

React Native & JavaScript

400+ Practice Questions

Multiple Choice Questions, True/False, and Code Correction across 16 topic groups — with a one-line explanation under every single answer.

MCQs: 224 | True/False: 112 | Code Correction: 64

1. Project Structure & Setup

Multiple Choice Questions (14)

1. Which folder typically holds reusable UI elements like buttons and cards?

A) assets/ B) components/ C) services/ D) constants/

Answer: B — components/ is the standard home for reusable UI pieces.

2. Where do API call functions (like axios instances) usually live?

A) components/ B) services/ C) assets/ D) hooks/

Answer: B — services/ centralizes network/API logic separate from UI.

3. Where should static values like color codes and font sizes be stored?

A) constants/ B) utils/ C) context/ D) app/

Answer: A — constants/ is meant for fixed config values used across the app.

4. Which folder holds custom hooks like useFetch or useAuth?

A) hooks/ B) services/ C) components/ D) assets/

Answer: A — hooks/ is the conventional folder for reusable custom hooks.

5. In Expo Router, which folder defines the navigation/screens structure?

A) components/ B) app/ C) utils/ D) constants/

Answer: B — Expo Router scans the app/ folder to build routes automatically.

6. Where should generic helper functions (e.g., formatDate) be placed?

A) utils/ B) assets/ C) context/ D) app/

Answer: A — utils/ is the standard place for small reusable helper functions.

7. Which folder holds Context API providers like AuthContext?

A) context/ B) services/ C) hooks/ D) constants/

Answer: A — context/ groups global state providers together.

8. What's a key benefit of a clean folder structure for teams?

A) Faster CPU B) Easier collaboration C) Smaller app size D) More RAM

Answer: B — Organized folders let multiple developers work without conflicts.

9. What is the assets/ folder primarily used for?

A) Store images/fonts B) Store API logic C) Store navigation D) Store global state

Answer: A — assets/ is for static files like images, icons, and fonts.

10. What happens to scalability without organized folders?

A) Improves B) Stays same C) Gets harder to manage D) Becomes automatic

Answer: C — Unorganized projects become increasingly hard to scale as files pile up.

11. Why keep services/ separate from components/?

A) To separate UI from data-fetching logic B) Required by React C) Makes app faster D) No real reason

Answer: A — Separating concerns keeps UI components focused only on rendering.

12. Which folder would contain a ThemeContext provider?

A) context/ B) constants/ C) assets/ D) app/

Answer: A — ThemeContext, like AuthContext, belongs alongside other global providers.

13. A new screen for product details should be added where in Expo Router?

A) components/ProductDetails.js B) app/product/[id].js C) services/ProductDetails.js D) utils/ProductDetails.js

Answer: B — Screens/routes belong inside app/, with dynamic segments in brackets.

14. Why is reusability listed as a benefit of organized folders?

A) Components in dedicated folders are easy to find and reuse B) It deletes duplicate code automatically C) It compresses files D) It's unrelated to folders

Answer: A — *When components are easy to locate, developers reuse them instead of rewriting.*

True / False (7)

1. The components/ folder is meant for reusable UI pieces.

Answer: True — *That's the very purpose of this folder.*

2. The assets/ folder is meant for API call functions.

Answer: False — *That's the job of services/, not assets/.*

3. A clean folder structure becomes more important as the app grows, not less.

Answer: True — *Small apps can survive messy structure, but it doesn't scale.*

4. Expo Router can build routes from files placed anywhere in the project, not just app/.

Answer: False — *Only files inside app/ are scanned for routing.*

5. Putting unrelated files in one folder makes debugging easier.

Answer: False — *Mixed responsibilities make it harder to locate relevant code.*

6. constants/ typically stores values like colors and spacing, not components.

Answer: True — *It's for static configuration values, not UI code.*

7. Good folder structure has no impact on how easily new developers understand a codebase.

Answer: False — *A predictable structure speeds up onboarding significantly.*

Code Correction (4)

1. logo.png placed directly in the project root

Fix: assets/images/logo.png

Images belong inside a dedicated assets folder, not the project root.

2. api_call.js sitting in root next to App.js

Fix: services/api_call.js

API logic should be grouped inside services/, separate from app entry files.

3. Button.js component placed inside utils/ folder

Fix: components/Button.js

UI components belong in components/, not utils/ which is for plain helper functions.

4. AuthContext.js placed inside components/ folder

Fix: context/AuthContext.js

Context providers belong in a dedicated context/ folder, not mixed with UI components.

2. Functional Components & JSX

Multiple Choice Questions (14)

1. What is a Functional Component in React Native?

- A) A class with render() B) A function returning JSX C) A CSS file D) A navigation config

Answer: B — *It's a plain JS function whose job is to describe and return the UI.*

2. Which tag replaces HTML's <div> in React Native?

- A) <Container> B) <View> C) <Box> D) <Section>

Answer: B — *<View> is RN's generic container component.*

3. Which tag replaces HTML's in RN?

- A) B) <Picture> C) <Image> D) <Photo>

Answer: C — *<Image> is RN's built-in component for displaying pictures.*

4. Which prop replaces HTML's onclick in RN?

- A) onClick B) onTap C) onPress D) onTouch

Answer: C — *RN uses onPress instead of the web's onClick.*

5. How is styling applied in JSX, unlike HTML's class attribute?

- A) style prop with object B) class prop C) css prop D) styleSheet attribute directly on tag

Answer: A — *Styles are objects passed through the style prop, not class names.*

6. What must wrap multiple JS variables/expressions inside JSX?

- A) [] B) {} C) () D) <>

Answer: B — *Curly braces let JSX evaluate plain JavaScript expressions.*

7. What happens if a JSX component returns two sibling elements without a wrapper?

- A) Works fine B) Throws an error/build fails C) Renders only the first D) Ignores both

Answer: B — *JSX compiles to single function-call trees, so siblings need one shared wrapper.*

8. Which syntax represents a comment inside JSX?

- A) // comment B) <!-- comment --> C) { /* comment */ } D) # comment

Answer: C — *JSX comments use curly braces around a JS-style block comment.*

9. Which JS construct is commonly used for conditional rendering in JSX?

- A) switch B) && or ternary ? : C) goto D) while

Answer: B — *&& and the ternary operator let you conditionally show JSX inline.*

10. What does a self-closing tag like <Image /> require?

- A) Nothing special B) A closing slash before > C) A separate closing tag always D) No attributes

Answer: B — *Self-closing tags must end with /> since they have no children.*

11. Which of these is valid JSX syntax for displaying a variable named title?

- A) {title} B) <%=title%> C) \${title} D) [title]

Answer: A — *Curly braces are JSX's way of inserting a JS expression's value.*

12. What replaces multiple sibling elements without adding an extra DOM node?

- A) <div> B) <Fragment> or <>...</> C) <Group> D) <Wrapper>

Answer: B — *A Fragment groups elements without rendering any extra wrapping node.*

13. In JSX, how do you pass a prop named name with value "Ali" to a component?

- A) name="Ali" B) name:="Ali" C) name->"Ali" D) set name="Ali"

Answer: A — *Props are passed just like HTML attributes, using name="value".*

14. Why are functional components generally preferred in modern React Native code?

- A) They're shorter and work well with Hooks B) They render faster always C) They don't need JSX D) They can't accept props

Answer: A — *Functional components pair naturally with Hooks and need less boilerplate.*

True / False (7)

1. JSX allows writing HTML tags like `<div>` directly in React Native.

Answer: False — *RN uses its own components like View/Text instead of HTML tags.*

2. JSX expressions inside `{}` are evaluated as JavaScript.

Answer: True — *Anything inside curly braces is treated as a JS expression.*

3. A functional component must always use the class keyword.

Answer: False — *Functional components are plain functions, not classes.*

4. JSX requires every opened tag to be properly closed.

Answer: True — *Unclosed tags will cause a build/compile error.*

5. Conditional rendering in JSX can use a ternary operator.

Answer: True — *A ternary (`? :`) is a common way to render one of two options.*

6. JSX comments use the same syntax as JavaScript line comments (`//`).

Answer: False — *JSX comments use `{/* ... */}` instead.*

7. style in React Native is applied as a CSS class name string.

Answer: False — *It's applied as a JavaScript object via the style prop.*

Code Correction (4)

1. `<Text>Hello name</Text>`

Fix: `<Text>Hello {name}</Text>`

Variables must be wrapped in curly braces to be evaluated, not printed as literal text.

2. `return <Text>A</Text><Text>B</Text>;`

Fix: `return <><Text>A</Text><Text>B</Text></>;`

Sibling elements need a single wrapping parent or Fragment to be returned together.

3. `<Image source="logo.png" />`

Fix: `<Image source={require('./logo.png')} />`

Local images must be loaded with require(), not passed as a plain string.

4. `<TouchableOpacity onClick={handlePress}>`

Fix: `<TouchableOpacity onPress={handlePress}>`

React Native uses onPress instead of the web's onClick.

3. JavaScript Functions

Multiple Choice Questions (14)

1. Which keyword declares a traditional function in JS?

A) func B) function C) def D) method

Answer: B — *function is the keyword used for standard function declarations.*

2. What does the return statement do?

A) Logs to console B) Stops execution and sends a value back C) Declares a variable D) Loops the function

Answer: B — *return ends the function call and hands a value back to the caller.*

3. What value does a function return if there's no return statement?

A) null B) 0 C) undefined D) false

Answer: C — *JavaScript defaults to undefined when no value is explicitly returned.*

4. Which is correct arrow function syntax?

A) (a,b) => a+b B) (a,b) -> a+b C) function(a,b) => a+b D) => (a,b) a+b

Answer: A — *Arrow functions use => between the parameters and the body.*

5. When does an arrow function NOT need an explicit return keyword?

A) When using curly braces B) When the body has no braces (implicit return) C) Never D) Always

Answer: B — *A brace-less arrow function body automatically returns its expression.*

6. What is a "function expression"?

A) A function assigned to a variable B) A function with no name ever C) A loop D) A class

Answer: A — *A function expression stores a function definition inside a variable.*

7. Which of these correctly calls a function named greet with argument "Hi"?

A) greet("Hi") B) greet->"Hi" C) call greet("Hi") D) greet:"Hi"

Answer: A — *Functions are invoked using parentheses containing their arguments.*

8. What's the result of calling a function that only has console.log() inside, no return?

A) Returns the logged string B) Returns undefined C) Throws an error D) Returns null

Answer: B — *Without a return statement, the call resolves to undefined.*

9. Which keyword is required before await inside a function?

A) defer B) async C) yield D) wait

Answer: B — *await is only valid inside functions declared with async.*

10. What does function add(a, b) {} (empty body) return when called?

A) 0 B) NaN C) undefined D) Error

Answer: C — *An empty function body has no return statement, so it returns undefined.*

11. What are the values passed into a function at call time usually called?

A) Arguments (parameters at definition time) B) Always arguments only C) Always constants D) Return values

Answer: A — *They're called parameters when defining the function, arguments when calling it.*

12. What's true about code written after a return statement in the same function?

A) It executes first B) It never executes C) It executes only in loops D) It executes only with async

Answer: B — *Code after return is unreachable — the function exits immediately.*

13. Which is the correct way to give parameter b a default value of 5?

A) function add(a, b=5) B) function add(a, b:5) C) function add(a, default b=5) D) function add(a, b==5)

Answer: A — *Default parameter values are written as param = value in the signature.*

14. Which function type lets you skip writing the word "function" entirely?

A) Function declaration B) Arrow function C) Named function expression D) IIFE only

Answer: B — *Arrow functions are written using => instead of the function keyword.*

True / False (7)

1. A function can have multiple return statements, but only one executes per call.

Answer: True — *Whichever return is reached first ends the function for that call.*

2. JavaScript functions always require a return statement.

Answer: False — *Without one, the function simply returns undefined.*

3. Arrow functions can be written without ever naming them like a regular function.

Answer: True — *Arrow functions are often anonymous and assigned to a variable.*

4. Calling a function and defining a function are the same step.

Answer: False — *Defining creates the function; calling actually executes it.*

5. `function add(a, b) { return a + b; }` is a valid function declaration.

Answer: True — *This is standard, correct function declaration syntax.*

6. Code after a return statement inside a function still runs.

Answer: False — *return immediately exits the function, skipping later code.*

7. Arrow functions with curly-brace bodies still need an explicit return keyword to send back a value.

Answer: True — *Braces switch to block-body syntax, which requires return explicitly.*

Code Correction (4)

1. `function add(a, b) { a + b; }`

Fix: `function add(a, b) { return a + b; }`

Without return, the calculated sum is never sent back to the caller.

2. `const square = (x) => { x * x };`

Fix: `const square = (x) => { return x * x; };`

A brace-bodied arrow function needs an explicit return to send back the value.

3. `function greet(name) console.log(name);`

Fix: `function greet(name) { console.log(name); }`

A function body must be wrapped in curly braces.

4. `add(2, 3) function;`

Fix: `function add(a, b) { return a + b; } add(2, 3);`

A function must be defined with the function keyword before it can be called.

4. Local vs. Remote Assets

Multiple Choice Questions (14)

1. Which function loads a local image bundled with the app?

A) fetch() B) require() C) import() only D) load()

Answer: B — *require() lets the bundler include the local image file at build time.*

2. What key is required in the source object for a network image?

A) src B) uri C) path D) href

Answer: B — *RN's <Image> expects a uri key for remote sources.*

3. Why must remote images have explicit width and height?

A) RN can't detect their size automatically B) It's optional styling C) Required only on Android D) Required only for GIFs

Answer: A — *Without a network image's real file, RN has no way to know its dimensions in advance.*

4. Can require() accept a dynamic variable as its path?

A) Yes always B) No, it must be a static string C) Only with template literals D) Only for PNGs

Answer: B — *The bundler must statically resolve the path at build time, not runtime.*

5. Which of these is correct syntax for a remote image source?

A) source="https://x.com/a.png" B) source={{uri: "https://x.com/a.png"}} C) source={url} D) source=uri("https://x.com/a.png")

Answer: B — *Remote sources must be objects containing a uri field.*

6. What happens if you forget width/height styles for a network image?

A) It auto-sizes like on web B) It may not render visibly C) App crashes always D) It becomes a local image

Answer: B — *Without explicit dimensions, the image can end up with no visible size.*

7. Local images are bundled into the app at what stage?

A) Runtime B) Build time C) Never D) After publishing only

Answer: B — *The bundler packages local images when the app is built.*

8. Which folder commonly stores local image assets?

A) services/ B) assets/images/ C) hooks/ D) context/

Answer: B — *Images conventionally live inside an assets/images folder.*

9. What's the correct way to reference a local logo file in <Image>?

A) source={require('./assets/images/logo.png')} B) source='./assets/images/logo.png' C) source={logo.png} D) src="logo.png"

Answer: A — *Local files must be wrapped in require() with a static relative path.*

10. Why can't you build a require() path using string concatenation with a variable?

A) Bundler can't statically resolve it B) It's too slow C) It's deprecated D) It only works with PNGs

Answer: A — *Metro needs to know exactly which files to bundle ahead of time.*

11. Which prop on <Image> defines its visual size?

A) dimension B) style (width/height) C) size D) scale

Answer: B — *Width and height are set through the style prop.*

12. For a network image, which part of source tells RN where to fetch from?

A) source.path B) source.uri C) source.link D) source.url

Answer: B — *uri is the field RN reads to know the image's network location.*

13. What's a key benefit of bundling local assets at build time?

A) Faster loading since no network call is needed B) Required for navigation C) Improves API speed D) Unrelated to performance

Answer: A — *Bundled images load instantly without a network request.*

14. Which would correctly load an icon that ships inside the app itself?

- A) `<Image source={{uri:'icon.png'}} />` B) `<Image source={require('./icon.png')} />` C) `<Image source="icon.png" />`
 D) `<Image uri="icon.png" />`

Answer: B — *A bundled local icon must be loaded through `require()`.*

True / False (7)

1. Network images need their source wrapped in an object with a `uri` key.

Answer: True — *This is how RN distinguishes a remote source from a local one.*

2. Local images can use a dynamic file path computed at runtime inside `require()`.

Answer: False — *`require()` paths must be static strings resolvable at build time.*

3. React Native can automatically detect a remote image's pixel dimensions like a web browser does.

Answer: False — *Remote dimensions must be set manually via style.*

4. Local images are bundled into the app during the build process.

Answer: True — *The bundler packages them in ahead of time.*

5. Forgetting to set width/height for a network image can make it invisible.

Answer: True — *Without a defined size, the image may render with zero visible area.*

6. `require()` and `import()` behave identically for dynamic paths.

Answer: False — *Neither supports dynamic/runtime-built paths for static bundling purposes.*

7. The `uri` key is only needed for local images, not remote ones.

Answer: False — *It's the opposite — `uri` is specifically for remote/network sources.*

Code Correction (4)

1. `<Image source="https://example.com/pic.jpg" />`

Fix: `<Image source={{ uri: "https://example.com/pic.jpg" }} style={{width:100,height:100}} />`
Remote sources must be objects with a `uri` key plus explicit dimensions.

2. `<Image source={require(imagePath)} />`

Fix: `<Image source={require('./assets/photo.png')} />`
`require()` needs a static string path, not a variable.

3. `<Image source={{uri:'photo.jpg'}} />`

Fix: `<Image source={{uri:'photo.jpg'}} style={{width:150,height:150}} />`
Remote images need explicit width/height or they may not display properly.

4. `<Image source='./assets/logo.png' />`

Fix: `<Image source={require('./assets/logo.png')} />`
Local image paths must be passed through `require()`, not as a plain string.

5. Navigation & Routing (Dynamic Routing in Expo Router)

Multiple Choice Questions (14)

1. What routing approach does Expo Router use?

- A) Config-based B) File-based C) XML-based D) Manual registration

Answer: B — *Routes are derived automatically from the app/ folder structure.*

2. Which symbol marks a dynamic segment in a route filename?

- A) {} B) [] C) () D) <>

Answer: B — *Square brackets, like [id].js, define a dynamic route parameter.*

3. Which hook reads a dynamic route parameter in Expo Router?

- A) useParams() B) useLocalSearchParams() C) useRouteData() D) useScreenParams()

Answer: B — *useLocalSearchParams() is Expo Router's hook for reading route params.*

4. A file named app/user/[userId].js matches which URL?

- A) /user/list B) /user/userId C) /user/42 D) /user

Answer: C — *Any value in that URL position is captured as the dynamic userId param.*

5. What does the parentheses syntax like (tabs) represent in Expo Router folder names?

- A) A dynamic route B) A route group that doesn't affect the URL path C) An error route D) A redirect

Answer: B — *Parenthesized folders organize routes without adding a path segment.*

6. Which file defines a shared layout/navigator for a folder of screens?

- A) index.js B) _layout.js C) config.js D) route.js

Answer: B — *_layout.js wraps and configures the screens inside its folder.*

7. What's the core idea behind file-based routing?

- A) Routes are manually registered in a single file B) Routes are automatically derived from the folder/file structure C) Routes require XML config D) Routes need a database

Answer: B — *The file system itself defines the app's navigation structure.*

8. If you navigate to /product/99, what will useLocalSearchParams().id return inside app/product/[id].js?

- A) undefined B) "99" C) 99 only as a number D) An array

Answer: B — *Route parameters are always returned as strings.*

9. Which navigator wraps screens for stack-style navigation in Expo Router?

- A) <Tabs> B) <Stack> C) <Drawer> D) <Switch>

Answer: B — *<Stack> provides the typical push/pop screen navigation.*

10. What happens if a route file/folder name doesn't match any Expo Router pattern?

- A) It's ignored for routing B) App crashes always C) It becomes the home route D) Nothing, errors are silent forever

Answer: A — *Unrecognized file patterns simply aren't treated as routes.*

11. Which is a correctly named dynamic route file for a blog post by slug?

- A) app/blog/slug.js B) app/blog/[slug].js C) app/blog/{slug}.js D) app/blog/(slug).js

Answer: B — *Dynamic segments require square brackets around the parameter name.*

12. What does useLocalSearchParams() return when there are multiple dynamic segments?

- A) Only the first one B) An object containing all matched parameters C) A single string D) Nothing

Answer: B — *It returns an object with a key for every matched dynamic segment.*

13. Why is file-based routing considered beginner-friendly?

- A) It requires extensive boilerplate B) Routes match the visible folder structure, making them predictable C) It needs no folders at all D) It disables navigation

Answer: B — *You can infer the routes just by looking at the file tree.*

14. Can nested folders inside app/ have their own dynamic segments?

A) No, only top-level B) Yes, e.g. app/shop/[category]/[item].js C) Only with class components D) Only on Android

Answer: B — *Dynamic segments can be nested at any folder depth.*

True / False (7)

1. Expo Router requires you to manually register each screen in a central config file.

Answer: False — *Routes are derived automatically from the folder structure.*

2. Square brackets in a filename create a dynamic route segment.

Answer: True — *That's exactly how dynamic routes like [id].js work.*

3. useLocalSearchParams() can only be used on the homepage route.

Answer: False — *It can be used on any route that needs to read URL parameters.*

4. _layout.js files can define a shared navigator for nested screens.

Answer: True — *That's their main purpose — wrapping and configuring nested routes.*

5. Route groups wrapped in parentheses, like (tabs), still appear as a path segment in the final URL.

Answer: False — *Parenthesized groups are excluded from the resulting URL path.*

6. A file app/post/[id].js will respond to a URL like /post/7.

Answer: True — *The bracketed segment matches any value, including "7".*

7. Expo Router only supports a single level of nested dynamic routes.

Answer: False — *Dynamic segments can be nested across multiple folder levels.*

Code Correction (4)

1. `import { useParams } from 'expo-router';`

Fix: `import { useLocalSearchParams } from 'expo-router';`

useParams doesn't exist in Expo Router; the correct hook is useLocalSearchParams.

2. `// File: app/product/{id}.js`

Fix: `// File: app/product/[id].js`

Dynamic segments use square brackets, not curly braces, in Expo Router filenames.

3. `const id = useLocalSearchParams.id;`

Fix: `const { id } = useLocalSearchParams();`

useLocalSearchParams is a hook that must be called as a function, then destructured.

4. `<Stack.Screen name="(tabs)/home" />`

Fix: `<Stack.Screen name="(tabs)" />`

You typically reference the route group itself, letting its own _layout.js handle nested screens.

6. Asynchronous Operations — Call Stack & Concurrency Model

Multiple Choice Questions (14)

1. What data structure does JS use to track active function calls?

- A) Queue B) Call Stack C) Heap D) Graph

Answer: B — *The Call Stack tracks which functions are currently running, in order.*

2. What does LIFO stand for in the context of the Call Stack?

- A) Last In, First Out B) Last In, Final Output C) Linear In, Final Out D) List In, First Out

Answer: A — *The most recently called function is the first one to finish and pop off.*

3. Why can JavaScript handle async tasks without freezing despite being single-threaded?

- A) It has multiple call stacks B) Its concurrency model offloads tasks to Web APIs C) It secretly uses multiple threads always D) It ignores async tasks

Answer: B — *Web APIs handle things like timers and network calls outside the main thread.*

4. What is the Event Loop's main job?

- A) Executing CSS B) Checking if the Call Stack is empty and running queued callbacks C) Compiling JS to machine code D) Managing the DOM only

Answer: B — *It continuously checks for an empty stack to schedule the next callback.*

5. Where does a setTimeout's callback go immediately after being called?

- A) Straight to Call Stack B) Web API environment C) Callback Queue immediately D) It runs synchronously right away

Answer: B — *The timer itself is handled outside the main thread, in the Web API layer.*

6. What happens to console.log("3") if placed after a setTimeout(...,0) call but before the script ends?

- A) It logs after the timeout callback B) It logs before the timeout callback C) Never logs D) Causes an error

Answer: B — *Synchronous code always finishes before any queued timeout callback runs.*

7. Why can't JS run two synchronous functions in parallel?

- A) Because it's single-threaded with one call stack B) Because functions are slow C) Because of network speed D) Because of HTML restrictions

Answer: A — *Only one thread executes JS code, so tasks run one at a time.*

8. What best describes a "microtask" in JS concurrency, e.g. for Promises?

- A) A task that always runs after macrotasks B) A queued callback processed with priority, ahead of regular callback queue tasks C) A CSS animation D) A type of variable

Answer: B — *Promise callbacks (microtasks) typically run before regular queued tasks.*

9. If the Call Stack is currently executing code, can the Event Loop interrupt it to run a queued task?

- A) Yes, anytime B) No, it waits until the stack is empty C) Only for Promises D) Only on mobile

Answer: B — *The event loop never interrupts a running function mid-execution.*

10. Which of these runs entirely on the Call Stack without ever touching Web APIs?

- A) setTimeout B) A simple synchronous loop like for(){ } C) fetch() D) axios.get()

Answer: B — *A plain synchronous loop has no async behavior, so it stays on the stack.*

11. What best describes "concurrency" in JavaScript's single-threaded model?

- A) True parallel execution B) The illusion of doing multiple things via the event loop and Web APIs C) Multiple call stacks running together D) Running on multiple CPU cores natively

Answer: B — *JS simulates multitasking by offloading work and resuming it later.*

12. Which of the following is offloaded to a Web API rather than run directly on the call stack?

- A) let x = 5; B) Network requests like fetch() C) if statements D) Variable declarations

Answer: B — *Network requests are handled outside the JS thread via Web APIs.*

13. What happens when a function call finishes executing?

A) It stays on the Call Stack forever B) It is popped off the Call Stack C) It moves to the DOM D) It becomes a Web API

Answer: B — *Completed calls are removed from the top of the stack.*

14. Why does the output order in async code sometimes surprise beginners?

A) Because async callbacks run later, after synchronous code finishes, even if written earlier in the file B) Because JS executes code randomly C) Because comments change execution order D) Because of typos only

Answer: A — *Code order in the file doesn't always match the actual execution order for async work.*

True / False (7)

1. JavaScript can run two pieces of synchronous code at exactly the same instant.

Answer: False — *Only one thread runs JS, so tasks execute strictly one at a time.*

2. The Call Stack works on a Last-In-First-Out basis.

Answer: True — *The most recently pushed call is the first to complete and pop off.*

3. setTimeout with a delay of 0ms runs its callback before any synchronous code below it.

Answer: False — *It's still queued through the Web API and runs only after the current code finishes.*

4. The Event Loop waits for the Call Stack to be empty before processing the next queued task.

Answer: True — *This avoids interrupting code that's already running.*

5. Web APIs allow tasks like network requests to run without blocking the main JS thread.

Answer: True — *That's exactly what keeps JS responsive during async work.*

6. JavaScript has multiple call stacks that run independently.

Answer: False — *It has a single call stack, since it's single-threaded.*

7. Microtasks (like Promise callbacks) are generally given priority over regular callback queue tasks.

Answer: True — *Microtasks are typically processed before the next regular queued task.*

Code Correction (4)

1. Assuming `setTimeout(fn, 0)` runs `fn` immediately, before later synchronous lines

Fix: Understand that `fn` always runs after the current synchronous code finishes
Even a 0ms timeout is queued through the Web API and only runs once the call stack clears.

2. Comment claiming "JS runs two functions in parallel using two threads"

Fix: "JS runs one function at a time on a single thread, using the event loop for concurrency"
JavaScript's core execution model is single-threaded, not multi-threaded.

3. `console.log(fetchData());` // expecting fetched data directly

Fix: `fetchData().then(data => console.log(data));`

An async function call doesn't return the final data synchronously; you must wait via `.then()` or `await`.

4. Belief that "the call stack can be paused mid-function to run a queued callback"

Fix: "A function on the call stack always runs to completion before any queued callback executes"

The event loop never interrupts a running function; it only acts when the stack is empty.

7. Environment Setup (Node.js for Expo/React Native)

Multiple Choice Questions (14)

1. What must be installed first to run any Expo/React Native CLI command?

- A) Android Studio B) Node.js C) Xcode D) Photoshop

Answer: B — *Node.js powers all the CLI tools and bundler used in development.*

2. Which command creates a brand-new Expo project?

- A) npm new expo B) npx create-expo-app my-app C) expo install new D) node expo-app

Answer: B — *This is the official command to scaffold a new Expo project.*

3. What tool bundles your JavaScript code for React Native apps?

- A) Webpack only B) Metro C) Babel alone D) Gradle

Answer: B — *Metro is React Native's standard JavaScript bundler.*

4. Which app lets you preview an Expo project on a physical phone without building natively?

- A) Expo Go B) Android Studio C) Xcode D) VS Code

Answer: A — *Expo Go runs your project directly without a native build step.*

5. Which tool is recommended on macOS for faster file watching during development?

- A) Watchman B) Finder C) Terminal D) Safari

Answer: A — *Watchman improves file-change detection speed during development.*

6. Which IDE/emulator tool is needed to run an iOS simulator?

- A) Android Studio B) Xcode (Mac only) C) Notepad D) Expo Go only

Answer: B — *iOS simulators require Xcode, which only runs on macOS.*

7. Why can't you run npx expo start without Node.js installed?

- A) Because npx itself is a Node.js tool B) Because of internet speed C) Because of RAM limits D) There's no real reason

Answer: A — *npx ships with Node.js and depends on it to run any command.*

8. What does npm primarily manage in an Expo project?

- A) Image compression B) Package/dependency installation C) Database queries D) CSS styling

Answer: B — *npm installs and manages the project's third-party packages.*

9. Which file lists a project's installed dependencies and scripts?

- A) App.js B) package.json C) babel.config.js D) README.md

Answer: B — *package.json records dependencies, scripts, and project metadata.*

10. What's the purpose of running npx expo start?

- A) Deletes the project B) Starts the Metro bundler/dev server C) Publishes to app stores directly D) Installs Node.js

Answer: B — *It launches the local development server for testing your app.*

11. Which platform requires Android Studio for emulator testing?

- A) Android B) iOS only C) Windows desktop apps D) None

Answer: A — *Android Studio provides the Android emulator used for testing.*

12. What's a basic requirement for developing React Native apps on any OS?

- A) Node.js and a code editor B) Only a smartphone C) A Mac is always required D) A paid Apple Developer account always

Answer: A — *These are the minimum essentials regardless of operating system.*

13. Why might npx be preferred over a global Expo CLI install?

- A) It always runs the latest version without needing a separate global install B) It's required for all JS code C) It avoids using Node.js entirely D) It only works offline

Answer: A — *npx fetches and runs the package fresh, avoiding outdated global installs.*

14. Which of these is NOT typically needed to start developing a basic Expo app?

A) Node.js B) A code editor C) A physical iPhone (an Android device, simulator, or Expo Go also work) D) npm

Answer: C — *You can develop and test without an iPhone specifically, using other options.*

True / False (7)

1. Node.js is required to run the Expo CLI and Metro bundler.

Answer: True — *Both tools are built on top of Node.js.*

2. You can fully develop and test an Expo app without installing Node.js.

Answer: False — *Every core dev tool depends on Node.js being installed.*

3. Expo Go lets you preview your app on a real device without a native build.

Answer: True — *It runs your JS bundle directly inside the Expo Go app.*

4. Xcode is required even for testing on Android emulators.

Answer: False — *Android testing relies on Android Studio, not Xcode.*

5. npx create-expo-app is the standard command to scaffold a new Expo project.

Answer: True — *It's the officially documented way to start a new project.*

6. Watchman is mandatory on all operating systems, including Windows.

Answer: False — *It's primarily recommended for macOS setups.*

7. package.json keeps track of a project's installed dependencies.

Answer: True — *It lists dependencies, devDependencies, and project scripts.*

Code Correction (4)

1. `npm install expo-cli -g` then `expo-cli create my-app` (outdated workflow)

Fix: `npx create-expo-app my-app`

The modern recommended way is npx create-expo-app, without a global CLI install.

2. `node create-expo-app my-app`

Fix: `npx create-expo-app my-app`

The command must be run through npx, not directly through node.

3. Assuming `npx expo start` publishes the app to the App Store

Fix: `npx expo start` only starts the local development server

Starting the dev server is for local testing, not publishing/releasing the app.

4. Running `expo start` without Node.js installed, expecting it to work

Fix: Install Node.js first, then run `npx expo start`

Every Expo CLI command depends on Node.js being installed on the machine.

8. SafeAreaView

Multiple Choice Questions (14)

1. What does SafeAreaView help your app avoid?

- A) Network errors B) Content hidden behind notches/status bars/gesture areas C) Slow renders D) State bugs

Answer: B — *It supplies inset data so layouts can avoid system UI overlap.*

2. Which package provides SafeAreaView?

- A) react-navigation B) react-native-safe-area-context C) expo-router D) axios

Answer: B — *This is the dedicated library for safe area handling in RN.*

3. Where should SafeAreaView typically be placed in the component tree?

- A) Inside a single button B) Near the root, wrapping the whole app C) Only inside modals D) It's not needed at all

Answer: B — *Placing it once at the root makes insets available everywhere.*

4. Which hook lets you read safe area inset values directly inside a component?

- A) useSafeAreaInsets() B) useInsets() C) useSafeArea() D) useEdges()

Answer: A — *This hook returns the current top/bottom/left/right inset values.*

5. Do Android devices with gesture navigation also need safe area handling?

- A) No, only iOS needs it B) Yes, gesture bars can overlap content too C) Only tablets need it D) Only old Android versions need it

Answer: B — *Gesture navigation bars on Android can also overlap app content.*

6. What problem occurs if SafeAreaView is used without a SafeAreaView ancestor?

- A) Nothing, it's optional B) Insets may be missing or inaccurate C) The app crashes immediately always D) Images stop loading

Answer: B — *SafeAreaView relies on a provider higher up to supply correct inset data.*

7. Which screen feature is SafeAreaView primarily designed to account for?

- A) Battery level B) Physical screen cutouts like notches and home indicators C) Wifi signal D) Audio output

Answer: B — *It's built specifically to handle these physical screen obstructions.*

8. What's the purpose of safe area insets?

- A) Measure space taken by system UI elements like notches/status bars B) Measure network latency C) Measure CPU usage D) Measure battery drain

Answer: A — *Insets tell you how much space the system UI occupies on each edge.*

9. Can SafeAreaView be used multiple times in different screens under one SafeAreaView?

- A) No, only once total B) Yes, each screen can use it as needed C) Only on iOS D) Only with class components

Answer: B — *One provider at the root can support SafeAreaView usage in many screens.*

10. Which kind of devices benefit most from SafeAreaView?

- A) Older flip phones B) Modern phones with notches, Dynamic Island, or gesture bars C) Desktop monitors D) None, it's purely cosmetic

Answer: B — *Modern phones with these screen features are exactly what it's designed for.*

11. If you don't use SafeAreaView at all, what's a likely visual issue?

- A) Slightly more padding everywhere B) A header or button could be hidden under a notch or status bar C) The app becomes faster D) Images become larger

Answer: B — *Without it, content might render in areas obstructed by system UI.*

12. Which of these best wraps the entire app in the typical setup?

- A) `<SafeAreaView><App content /></SafeAreaView>` B) `<App content><SafeAreaView /></App content>`
C) SafeAreaView is placed inside every Text element D) It replaces the Stack navigator

Answer: A — *The provider should wrap around all of the app's content, near the root.*

13. What does `useSafeAreaInsets()` typically return?

- A) A single number B) An object with top, bottom, left, right inset values C) A boolean D) A color string

Answer: B — *It gives separate measurements for each side of the screen.*

14. Why is `SafeAreaProvider` placed once near the root rather than per-screen?

- A) So every nested screen can access consistent inset data through context B) Because it only works once per app
C) Because it disables itself after first use D) There's no specific reason

Answer: A — *A single root-level provider makes the data available everywhere below it.*

True / False (7)

1. `SafeAreaProvider` is only useful for iPhones with a notch.

Answer: False — *It also helps on Android devices with gesture navigation bars.*

2. `SafeAreaProvider` should generally be placed near the root of the app.

Answer: True — *That ensures all nested screens can access accurate inset data.*

3. `useSafeAreaInsets()` returns inset values like top, bottom, left, and right.

Answer: True — *It gives a separate measurement for each screen edge.*

4. `SafeAreaView` works correctly even with no `SafeAreaProvider` anywhere in the tree, with no downsides.

Answer: False — *Without a provider, its inset values can be missing or inaccurate.*

5. Gesture-based Android navigation bars can also overlap app content without proper safe area handling.

Answer: True — *This is one of the main reasons Android also needs safe area support.*

6. `SafeAreaProvider` directly fixes slow network requests.

Answer: False — *It has nothing to do with networking; it's about screen layout.*

7. You generally only need to set up `SafeAreaProvider` once per app, not once per screen.

Answer: True — *A single provider near the root covers the entire app.*

Code Correction (4)

1. `<SafeAreaView style={{flex:1}}>{content}</SafeAreaView>` with no provider anywhere

Fix: Wrap the whole app in `<SafeAreaProvider>` first, then use `SafeAreaView` inside it
SafeAreaView depends on inset data supplied by an ancestor SafeAreaProvider.

2. `<SafeAreaProvider>` placed individually inside every single screen component

Fix: Place a single `<SafeAreaProvider>` once near the root `_layout.js`
One provider at the root is enough; it doesn't need to be repeated on every screen.

3. Assuming `useSafeAreaInsets()` returns just one number for "padding"

Fix: `const { top, bottom, left, right } = useSafeAreaInsets();`
It returns an object with separate inset values for each side of the screen.

4. Skipping `SafeAreaProvider` entirely because "the app only targets Android"

Fix: Include `SafeAreaProvider` regardless of target platform
Many modern Android devices use gesture navigation bars that also require safe area handling.

9 & 10. Lifecycle of an Async API Call and Why It Matters

Multiple Choice Questions (14)

1. When `axios.get()` is called, where does the actual network request execute?

- A) Call Stack B) Web API environment C) Local Storage D) Render tree

Answer: B — *The network call is handed off outside JS's single main thread.*

2. What gets placed into the Callback/Microtask Queue once a network response arrives?

- A) The original `axios.get()` call again B) The `.then()` callback (or resumed `await` code) C) The whole app D) Nothing

Answer: B — *Only the code waiting on the result is queued once the response is ready.*

3. What must be true before the Event Loop pushes a queued callback onto the Call Stack?

- A) The stack must be empty B) The network must be fast C) The app must be in the foreground D) There must be no other code at all, ever

Answer: A — *The event loop always waits for the current call stack to clear first.*

4. Why does an async API call not freeze the UI?

- A) Because the network call is offloaded to the Web API, not run synchronously on the call stack B) Because phones are fast C) Because of caching only D) Because `await` blocks the UI thread by design

Answer: A — *Offloading the wait keeps the main thread free to handle user interaction.*

5. Given: `console.log("A"); axios.get(url).then(()=>console.log("B")); console.log("C");` — what's the typical output order?

- A) A, B, C B) A, C, B C) B, A, C D) C, B, A

Answer: B — *Synchronous lines run first; the `.then()` callback fires only after the response arrives.*

6. What's the benefit of this non-blocking model for user experience?

- A) Users can keep scrolling/tapping while data loads B) It removes all bugs C) It makes API calls instant D) It disables loading spinners

Answer: A — *The interface stays responsive while data is fetched in the background.*

7. At what point does `res.data` actually become accessible in a `.then()` callback?

- A) Immediately when `axios.get()` is called B) Only after the response arrives and the callback runs C) Before the request is even sent D) Never, it's always undefined

Answer: B — *The data only exists once the server has actually responded.*

8. Which best describes "Web API" in this context?

- A) A REST API you must build yourself B) Browser/Node-provided functionality (like network calls, timers) that runs outside the main JS thread C) A type of HTML element D) A CSS feature

Answer: B — *It's the environment-provided layer handling things like networking and timers.*

9. If you try to log a variable assigned inside a `.then()` callback immediately after calling `axios.get()`, outside the callback, what do you typically get?

- A) The correct data B) undefined, because the assignment hasn't happened yet C) An error always D) The previous page's data

Answer: B — *The assignment only happens later, once the response actually arrives.*

10. What's the correct general order of the async lifecycle for `axios.get()`?

- A) Call Stack → Web API → Callback Queue → Event Loop → Call Stack B) Call Stack only C) Web API → Call Stack → Web API again D) Callback Queue → Web API → Call Stack

Answer: A — *This sequence describes how the request is offloaded and later resumed.*

11. Why is this lifecycle described as "non-blocking"?

- A) Because it literally has no code B) Because the main thread isn't paused waiting for the network response C) Because it always finishes instantly D) Because of multiple threads

Answer: B — *The main thread keeps running other code instead of waiting idle.*

12. What would happen to the app's responsiveness if every API call ran synchronously and blocked the thread?

A) No difference B) The UI would freeze until each call finished C) The app would run faster D) Nothing changes for the user

Answer: B — *A blocking call would lock up button taps and scrolling until it finished.*

13. Which part of the lifecycle ensures the callback doesn't run while other code is still executing?

A) The Web API B) The Event Loop's check that the Call Stack is empty C) The HTML parser D) The CSS engine

Answer: B — *The event loop only schedules the callback once the stack is clear.*

14. In `async function getData(){ const res = await axios.get(url); }`, what does `await` effectively do?

A) Pauses only that function until the Promise resolves, without blocking the rest of the app B) Freezes the entire app C) Cancels the request D) Runs the request twice

Answer: A — *Only the awaiting function pauses — everything else keeps running.*

True / False (7)

1. The `.then()` callback for an API call executes synchronously, the instant `axios.get()` is called.

Answer: False — *It only runs later, once the network response actually arrives.*

2. The actual network request is handled outside JavaScript's single Call Stack, via the Web API.

Answer: True — *This is what keeps the main thread free while waiting.*

3. The Event Loop only pushes a queued callback once the Call Stack is empty.

Answer: True — *It never interrupts code that's currently running.*

4. Because of this async model, users can still interact with the app while data is loading.

Answer: True — *The UI thread isn't blocked while the network call is in progress.*

5. If you log a variable right after calling `axios.get()` (outside any callback), it will already contain the fetched data.

Answer: False — *The data is assigned later, inside the callback, once the response arrives.*

6. `await` pauses the entire application, not just the function it's used in.

Answer: False — *Only the specific async function pauses; the rest of the app keeps running.*

7. The general lifecycle order is: Call Stack → Web API → Callback Queue → Event Loop → Call Stack.

Answer: True — *This is the standard path an async call follows.*

Code Correction (4)

1. `let data; axios.get(url).then(res => data = res.data); console.log(data);`

Fix: `axios.get(url).then(res => console.log(res.data));`

The log must happen inside the callback, after the data has actually arrived.

2. `const data = axios.get(url); // expecting data directly`

Fix: `const res = await axios.get(url); const data = res.data; // inside an async function`
`axios.get()` returns a Promise, not the data itself — you need `await` or `.then()`.

3. `function getData(){ axios.get(url).then(res=>{ return res.data; }); } console.log(getData());`

Fix: `async function getData(){ const res = await axios.get(url); return res.data; }`
`getData().then(data=>console.log(data));`

The outer function must properly await or return the inner Promise to expose the data.

4. Belief that "the UI freezes while `axios.get()` waits for a response"

Fix: "The UI stays responsive because the request runs on the Web API, not the main call stack"

That's the entire point of JavaScript's non-blocking concurrency model for network calls.

11. The Role of AuthContext (Context API)

Multiple Choice Questions (14)

1. What React feature does AuthContext rely on?

- A) Context API B) Redux only C) Local Storage D) CSS variables

Answer: A — *AuthContext is built using React's built-in Context API.*

2. What problem does Context API solve for auth state?

- A) Prop drilling through many components B) Slow networks C) Image compression D) CSS conflicts

Answer: A — *It lets data skip directly to any component without manual prop passing.*

3. What does AuthProvider typically wrap?

- A) A single button B) The whole app (or large sections of it) C) Only the login screen D) Nothing, it's unused

Answer: B — *It needs to wrap everything that should have access to auth state.*

4. Which hook lets a component access AuthContext's value easily?

- A) useState() B) useAuth() (a custom hook wrapping useContext) C) useEffect() D) useReducer() only

Answer: B — *useAuth() is the common convenience wrapper around useContext(AuthContext).*

5. What does AuthContext.Provider's value prop usually contain?

- A) CSS classes B) user state plus login/logout functions C) Image URIs D) Navigation history only

Answer: B — *This is exactly the data every consuming component receives.*

6. Why use createContext() at the top of AuthContext.js?

- A) To create the Context object that Provider and useContext rely on B) To start the Metro bundler C) To define routes D) To create a database connection

Answer: A — *createContext() is the first step in setting up any Context.*

7. What's the correct way to read the current user inside a deeply nested component?

- A) Pass user as a prop through every parent B) Call useAuth() (or useContext(AuthContext)) directly C) Read AuthContext.user directly D) Import the login screen

Answer: B — *Context lets any nested component subscribe directly to the value.*

8. What does calling logout() typically do inside AuthProvider?

- A) Deletes the app B) Sets user state back to null C) Restarts Metro D) Clears all images

Answer: B — *Logging out clears the stored user so the app treats them as signed out.*

9. Without Context API, how would auth data normally have to be shared with deeply nested components?

- A) Via prop drilling through every intermediate component B) Automatically, with no extra code C) It couldn't be shared at all D) Only through global variables on window

Answer: A — *Each intermediate component would need to forward the data manually.*

10. What does useContext(AuthContext) trigger when the context value changes?

- A) Nothing B) A re-render of components consuming that context C) A page reload D) An app crash

Answer: B — *Consumers automatically re-render whenever the provided value updates.*

11. Where is AuthProvider typically placed in the component tree?

- A) Inside a single screen only B) Near the root, often inside the root _layout.js C) Inside a Text component D) It doesn't need to be rendered at all

Answer: B — *Placing it near the root makes auth data available app-wide.*

12. What's a key benefit of using a custom hook like useAuth() instead of useContext(AuthContext) everywhere?

- A) Cleaner, more readable code across components B) It's required by React C) It disables Context entirely D) It removes the need for a Provider

Answer: A — *It shortens repeated boilerplate and keeps usage consistent.*

13. If AuthContext is created with createContext() but no Provider wraps the app, what happens when a component calls useContext(AuthContext)?

A) It works perfectly with full data B) It gets the default value passed to createContext() (often undefined) C) The app won't compile D) It auto-creates a Provider

Answer: B — *Without a Provider, components just receive the Context's default value.*

14. Which best describes "global state" in the context of AuthContext?

A) State accessible from many components without manual prop passing B) A type of CSS variable C) A kind of local storage D) A database table

Answer: A — *Global state is shared data reachable from anywhere in the component tree.*

True / False (7)

1. AuthContext avoids the need to manually pass auth data as props through every intermediate component.

Answer: True — *That's exactly the prop-drilling problem Context API solves.*

2. AuthContext.Provider's value prop usually includes the current user and login/logout functions.

Answer: True — *This is the typical shape of data passed down.*

3. You can read a Context's current value directly via dot notation like AuthContext.user.

Answer: False — *You must use useContext (or a hook wrapping it) to read the live value.*

4. useAuth() is commonly just a small wrapper around useContext(AuthContext) for convenience.

Answer: True — *It saves repeating useContext(AuthContext) everywhere.*

5. AuthProvider must be placed inside every single screen separately to work correctly.

Answer: False — *One provider near the root is enough for the whole app.*

6. Calling the logout function typically sets the stored user state back to null.

Answer: True — *This effectively marks the user as signed out.*

7. Context API is unrelated to React's re-rendering system.

Answer: False — *Consumers re-render automatically when the context value changes.*

Code Correction (4)

1. `const user = AuthContext.user;`

Fix: `const { user } = useAuth();`

Context values must be read with useContext (or a hook wrapping it), not direct property access.

2. `<AuthContext.Provider>{children}</AuthContext.Provider>` (no value prop passed)

Fix: `<AuthContext.Provider value={{ user, login, logout }}>{children}</AuthContext.Provider>`

Without a value prop, consuming components won't receive the actual auth data.

3. Passing user manually as a prop through five nested components to reach a deeply nested screen

Fix: Wrap the app in AuthProvider and call useAuth() directly inside that nested screen

Context API exists specifically to avoid this kind of repetitive prop drilling.

4. `function useAuth() { return AuthContext; }`

Fix: `function useAuth() { return useContext(AuthContext); }`

The custom hook must call useContext to actually subscribe to and return the current value.

12. _layout.js and Protected Routes

Multiple Choice Questions (14)

1. What decides which screens are rendered in a protected _layout.js setup?

- A) Random selection B) The current authentication state (user) C) Device battery level D) Screen file size

Answer: B — *The auth state is the deciding factor for which Stack is shown.*

2. When user is null, which screen should typically be reachable?

- A) All protected screens B) Only the login screen C) None at all, app crashes D) A random screen

Answer: B — *Only login should be accessible until the user authenticates.*

3. After a successful login() call, what usually happens to the rendered Stack?

- A) Nothing changes B) It re-renders to expose the protected (tabs) group instead of login C) The app closes D) It shows an error page

Answer: B — *The layout reacts to the updated auth state and swaps the visible Stack.*

4. Why is checking auth state at the layout level better than checking it inside every screen?

- A) It centralizes the logic in one place instead of repeating it everywhere B) It's required by Expo C) It disables navigation entirely D) There's no real benefit

Answer: A — *One central check avoids duplicating the same logic across many screens.*

5. What import is typically needed to build a Stack navigator inside _layout.js?

- A) import { Stack } from 'expo-router' B) import Stack from 'react' C) import { Stack } from 'react-native' D) No import needed

Answer: A — *Stack comes from the expo-router package.*

6. If both protected and login routes are rendered unconditionally with no check, what's the risk?

- A) None, it's safer B) A logged-out user could potentially reach protected screens C) The app becomes faster D) Images stop loading

Answer: B — *Without a condition, nothing actually blocks access to protected screens.*

7. Which hook is commonly used inside _layout.js to read the current user?

- A) useAuth() B) useImage() C) useStyle() D) useRoute() only

Answer: A — *useAuth() supplies the current auth state used for the conditional check.*

8. What's the simplest condition to check if nobody is logged in?

- A) if (user) B) if (!user) C) if (user === true) D) if (user.length)

Answer: B — *!user is true when user is null or undefined, meaning nobody's logged in.*

9. What does options={{ headerShown: false }} typically do on a Stack.Screen?

- A) Hides the default navigation header for that screen B) Deletes the screen C) Disables the screen entirely D) Changes the screen's route name

Answer: A — *It's a common option to hide the default header bar.*

10. What triggers the layout to re-render with different screens in a protected-routes pattern?

- A) A change in the auth state (e.g., user becoming non-null) B) Changing the phone's wallpaper C) Restarting the Metro bundler D) Nothing ever changes it

Answer: A — *The layout reacts whenever the auth state itself changes.*

11. Why might you use a Redirect component instead of, or alongside, conditional rendering for protected routes?

- A) To explicitly send a user to a particular route based on auth state B) To delete a route C) To disable the app D) To install a package

Answer: A — *A Redirect can programmatically navigate based on the current condition.*

12. What's a typical name for the wrapped protected section in many Expo Router auth examples?

- A) (tabs) B) (public) C) (error) D) (none)

Answer: A — *(tabs) is a common route group name for the main authenticated app.*

13. If you forget to wrap your protected-route check in the root `_layout.js`, what's the consequence?

- A) Better performance B) Screens might be reachable regardless of login status C) The app becomes a website D) Nothing, it's purely cosmetic

Answer: B — *Without the check, protected content isn't actually protected.*

14. What's the role of the user variable from `useAuth()` in this whole pattern?

- A) It acts as the single source of truth for whether someone is authenticated B) It styles the buttons C) It stores images D) It's unrelated to navigation

Answer: A — *Everything in this pattern hinges on the current value of user.*

True / False (7)

1. The root `_layout.js` can use the auth state to decide which Stack of screens to render.

Answer: True — *That's the core mechanism behind this protected-routes pattern.*

2. Each individual screen must independently re-check the login status for protected routes to work.

Answer: False — *The check happens once, centrally, in the layout.*

3. Once user becomes non-null after login, the layout can re-render to show the protected screens.

Answer: True — *The change in auth state triggers a re-render with different screens.*

4. Protected routes guarantee complete security with zero risk, even without any backend validation.

Answer: False — *Client-side checks alone aren't a substitute for proper backend security.*

5. `options={{ headerShown: false }}` hides the default header bar for a screen.

Answer: True — *It's a commonly used `Stack.Screen` option.*

6. If both login and protected screens are always rendered with no conditional check, the protection is effectively missing.

Answer: True — *Without a condition, nothing actually restricts access.*

7. `useAuth()` is unrelated to determining what's shown in a protected-routes layout.

Answer: False — *It supplies exactly the auth state the layout's condition depends on.*

Code Correction (4)

1. Both `Stack.Screen` entries rendered unconditionally with no auth check

Fix: Wrap in `if (!user) { return <Stack><Stack.Screen name="login" /></Stack>; }` before the protected `Stack`

Without a conditional check, both protected and login routes are always reachable.

2. `const user = useAuth;` (missing parentheses)

Fix: `const { user } = useAuth();`

`useAuth` is a function/hook and must be called with parentheses, then destructured.

3. Checking auth status separately inside every single protected screen component

Fix: Check it once centrally inside the root `_layout.js`

Centralizing the check in the layout avoids repeating the same logic across many screens.

4. `if (user) { return <Stack><Stack.Screen name="login" /></Stack>; }` (inverted logic)

Fix: `if (!user) { return <Stack><Stack.Screen name="login" /></Stack>; }`

The login screen should show when there is no user, not when there is one — the condition was backwards.

13. Named Export vs. Export Default

Multiple Choice Questions (14)

1. How many default exports can a single file have?

- A) Unlimited B) Exactly one C) Exactly two D) Zero, always

Answer: B — *A file can only designate one value as its default export.*

2. How do you import a named export called formatDate from utils.js?

- A) import formatDate from './utils' B) import { formatDate } from './utils' C) import * formatDate from './utils' D) require formatDate('./utils')

Answer: B — *Named exports must be imported using curly braces and the exact name.*

3. When importing a default export, must the local name match the original export name?

- A) Yes, exactly B) No, any name can be used C) Only for components D) Only for functions

Answer: B — *Default exports have no fixed name; you choose it at import time.*

4. Which export style is best for a file that only exports one main React component?

- A) Named export B) Export default C) Both required always D) Neither

Answer: B — *A single main export is the classic use case for export default.*

5. Which export style is best for a file with several small utility functions?

- A) Export default for each B) Named exports C) No exports needed D) Default exports only

Answer: B — *Named exports let a file expose multiple distinct values.*

6. What happens if you try to write two export default statements in one file?

- A) Both work fine B) A SyntaxError occurs C) Only the last one is used silently D) Only the first one is used silently

Answer: B — *Only one default export is allowed per file, by language rule.*

7. Which syntax correctly performs a named export of a constant called PI?

- A) export default const PI = 3.14; B) export const PI = 3.14; C) export PI = 3.14; D) module.exports PI = 3.14;

Answer: B — *This is the standard way to write a named export of a constant.*

8. How do you import multiple named exports, add and subtract, from the same file?

- A) import add, subtract from './math' B) import { add, subtract } from './math' C) import [add, subtract] from './math' D) import add; import subtract;

Answer: B — *Multiple named exports are imported together inside one set of curly braces.*

9. Which of these correctly imports a default export under a custom local name?

- A) import MyButton from './Button' B) import { MyButton } from './Button' C) import default MyButton from './Button' D) import Button as default from './Button'

Answer: A — *Default imports skip curly braces and can use any chosen local name.*

10. What's a common real-world example of a default export?

- A) A single React screen/component file B) A file with 10 unrelated helper functions C) A constants file with many values D) A CSS file

Answer: A — *Single-purpose component files commonly use a default export.*

11. What's a common real-world example of named exports?

- A) A single button component B) A utils.js file exporting several helper functions C) A single App.js entry file D) An image file

Answer: B — *Utility files with several related helpers typically use named exports.*

12. Which statement about named exports is true?

- A) They require curly braces when importing B) They can never be combined in one file with a default export C) They must always be anonymous D) They can't export functions

Answer: A — *Named imports always use curly braces around the exact export name.*

13. Can a single file have both a default export and several named exports together?

A) No, never B) Yes, that's allowed C) Only in TypeScript D) Only with classes

Answer: B — *JavaScript modules support mixing one default export with multiple named ones.*

14. What happens if you write `import { Button } from './Button'`; but `Button.js` only has `export default function Button(){...}`?

A) No error, it works fine B) `Button` ends up undefined, since named import syntax doesn't match a default export C) It silently renames `Button` D) It auto-converts to default import

Answer: B — *Named-import syntax can't retrieve a default export; the import won't match.*

True / False (7)

1. A single JS file can have multiple named exports.

Answer: True — *Named exports support exporting many distinct values from one file.*

2. A single JS file can have multiple default exports.

Answer: False — *Only one default export per file is allowed.*

3. Default exports must be imported using curly braces.

Answer: False — *Default exports are imported without curly braces.*

4. Named exports must be imported using the exact same name they were exported with (unless renamed with "as").

Answer: True — *The import name must match unless you explicitly rename it.*

5. A file can combine one default export with several named exports.

Answer: True — *Mixing both export styles in one file is valid JavaScript.*

6. Default exports are best suited for files that export many unrelated utility functions.

Answer: False — *Named exports are better suited for that use case.*

7. Using the wrong curly-brace syntax to import a default export (e.g. `import { Button }` when it's a default export) will not correctly retrieve it.

Answer: True — *Default and named imports use different syntax that aren't interchangeable.*

Code Correction (4)

1. `export default const add = (a,b) => a+b; export default const subtract = (a,b) => a-b;`

Fix: `export const add = (a,b) => a+b; export const subtract = (a,b) => a-b;`

A file can only have one default export, so both should become named exports instead.

2. `import Button from './utils';` // where `utils.js` only has `export const formatDate = () => {}`

Fix: `import { formatDate } from './utils';`

formatDate is a named export, so it must be imported with curly braces using its exact name.

3. `export default function Button() {} ... import { Button } from './Button';`

Fix: `import Button from './Button';`

Default exports are imported without curly braces, using any name you choose.

4. `export const default PI = 3.14;`

Fix: `export const PI = 3.14;`

"default" isn't a valid modifier combined with const in a named export statement.

14. useState vs. useEffect

Multiple Choice Questions (14)

1. What does useState(0) return?

- A) Just 0 B) An array: [currentValue, setterFunction] C) An object D) A Promise

Answer: B — *It returns a pair you typically destructure into a value and its setter.*

2. What triggers a component to re-render after using useState?

- A) Calling the setter function B) Calling console.log C) Refreshing the whole app manually D) Nothing automatically

Answer: A — *Calling the setter is what tells React to re-render with the new value.*

3. What does an empty dependency array [] in useEffect mean?

- A) Run on every render B) Run only once, after the initial mount C) Never run D) Run only when unmounting

Answer: B — *React only treats the "first render" condition as true once, at mount.*

4. What happens if you omit the dependency array entirely in useEffect?

- A) It runs once only B) It runs after every render C) It never runs D) It throws an error always

Answer: B — *Without an array, there's no condition limiting how often it runs.*

5. Which hook is best suited for fetching data when a component first mounts?

- A) useState alone B) useEffect with an empty dependency array C) useRef D) useContext

Answer: B — *This is the standard pattern for an on-mount data fetch.*

6. What's a common bug when useEffect's dependency array is missing for an API call inside it?

- A) Nothing bad happens B) Infinite re-fetching loop C) The component never renders D) TypeScript error always

Answer: B — *Updating state inside the effect re-triggers the effect endlessly.*

7. Can useEffect's dependency array include a state variable so it re-runs when that variable changes?

- A) No, never B) Yes, e.g. [count] re-runs the effect whenever count changes C) Only with class components D) Only for navigation

Answer: B — *Listing a value in the array makes the effect re-run when it changes.*

8. What is "state" in the context of useState?

- A) Data that persists and can change over a component's lifetime, triggering re-renders when updated B) A CSS property C) A type of network request D) A fixed constant that can't change

Answer: A — *State represents component data that can change and drives re-renders.*

9. Which of these correctly declares state for a loading boolean, initially true?

- A) const [loading, setLoading] = useState(true); B) const loading = useState(true); C) let loading = true; D) const loading = true;

Answer: A — *This correctly destructures both the value and its setter function.*

10. Why shouldn't you call useState or useEffect conditionally inside an if statement?

- A) It violates React's Rules of Hooks and can cause bugs B) It's just a style preference C) It's required for performance D) There's no real issue

Answer: A — *Hooks must be called in the same order on every render, unconditionally.*

11. What does the cleanup function returned from useEffect typically do?

- A) Runs cleanup logic (e.g., clearing a timer) before the effect re-runs or the component unmounts B) Starts the app C) Deletes all state D) Renders the next screen

Answer: A — *It's used to undo side effects, like clearing intervals or subscriptions.*

12. If a useEffect depends on [userId], when does it re-run?

- A) Never B) Every time userId changes between renders C) Only once ever, regardless of userId D) Only when the app restarts

Answer: B — *The effect re-fires whenever the listed dependency's value changes.*

13. What's the safest way to update state based on its previous value?

A) `setCount(count + 1)` only B) `setCount(prevCount => prevCount + 1)` C) `count = count + 1` D) `useEffect(() => count + 1)`

Answer: B — *The updater-function form always works off the latest state value.*

14. Which of these best describes a "side effect" in React?

A) Pure rendering logic only B) Code that interacts with something outside the component, like data fetching or subscriptions C) A CSS animation only D) A type of prop

Answer: B — *Side effects are operations that reach outside the component's own rendering.*

True / False (7)

1. Calling the `useState` setter function causes the component to re-render with the new value.

Answer: True — *That's the mechanism by which state updates appear on screen.*

2. `useEffect` with no dependency array runs only once, just like an empty array.

Answer: False — *With no array, it runs after every single render, not just once.*

3. An effect with `[]` as its dependency array runs exactly once, right after the component mounts.

Answer: True — *An empty array means there's nothing that could trigger a re-run.*

4. Forgetting the dependency array on an effect that updates state used inside it can cause an infinite loop.

Answer: True — *Each state update can re-trigger the effect again indefinitely.*

5. `useState` can only store numbers, not arrays or objects.

Answer: False — *`useState` can hold any JavaScript value, including arrays and objects.*

6. `useEffect` can be used to fetch data when a component mounts.

Answer: True — *This is one of its most common use cases.*

7. Hooks like `useState` and `useEffect` should be called conditionally inside if-statements for best practice.

Answer: False — *Hooks must always be called unconditionally, in the same order every render.*

Code Correction (4)

1. `useEffect(() => { axios.get('/data').then(res=>setData(res.data)); });` (no dependency array)

Fix: `useEffect(() => { axios.get('/data').then(res=>setData(res.data)); }, []);`
Without `[]`, the effect re-runs after every render, including the one triggered by `setData`.

2. `const [count, setCount] = useState;` (missing function call)

Fix: `const [count, setCount] = useState(0);`
`useState` must be called as a function with an initial value, not referenced without parentheses.

3. `if (isLoggedIn) { useEffect(() => {...}); }`

Fix: `useEffect(() => { if (isLoggedIn) { /* ... */ } }, [isLoggedIn]);`
Hooks must be called unconditionally at the top level; the condition belongs inside the effect.

4. `setCount(count + 1); setCount(count + 1);` // expecting count to increase by 2

Fix: `setCount(prev => prev + 1); setCount(prev => prev + 1);`
Using the updater-function form ensures each call works off the latest state rather than a stale snapshot.

15. Navigation Hierarchy: `_layout.js` as the Root Configuration

Multiple Choice Questions (14)

1. Why is the root `_layout.js` called the "Root Configuration"?

- A) Because it's the first layout every screen in the app passes through B) Because it must contain only styles C) Because it can't include providers D) Because it's optional

Answer: A — *Every navigation path in the app passes through it first.*

2. What can be defined once in the root `_layout.js` and used by every nested screen?

- A) Context Providers and the top-level navigator B) Only colors C) Only fonts D) Nothing reusable

Answer: A — *These app-wide concerns are set up once and cascade to all screens.*

3. Can a nested folder, like `(tabs)/`, have its own `_layout.js`?

- A) No, only the root can have one B) Yes, creating a layout hierarchy C) Only if there are no dynamic routes D) Only for class components

Answer: B — *Nested folders can define their own scoped layout, building a hierarchy.*

4. If `AuthProvider` wraps the `Stack` inside the root `_layout.js`, which screens can access the auth context?

- A) Only the root layout itself B) Every nested screen in the app C) No screens, only components D) Only screens named "auth"

Answer: B — *Context cascades down to all components nested beneath it.*

5. What does a typical root `_layout.js` usually return?

- A) A single `Text` element B) A navigator (like `Stack`) wrapped in any global providers needed C) An empty fragment always D) A CSS stylesheet

Answer: B — *It defines the navigator and wraps it with app-wide providers.*

6. Why shouldn't global providers like `SafeAreaProvider` be placed inside an individual screen file instead of the root layout?

- A) Because then only that one screen would have access to them B) Because it's against React rules entirely C) Because screens can't have any imports D) There's no real reason

Answer: A — *Other screens outside that one file wouldn't benefit from the provider.*

7. What best describes the relationship between a root layout and a nested `(tabs)/_layout.js`?

- A) The nested layout is wrapped by/sits underneath the root layout B) They are unrelated and independent C) The nested layout overrides the root layout entirely D) Only one of them can exist

Answer: A — *The nested layout operates within the structure the root layout sets up.*

8. Which of these would likely break the intended architecture?

- A) Defining `AuthProvider` in `app/_layout.js` B) Defining a `Stack` navigator with multiple `Stack.Screen` entries inside an individual screen file like `home.js` C) Using `SafeAreaProvider` once at the root D) Using `_layout.js` to wrap shared UI

Answer: B — *Navigators belong in layout files, not inside individual leaf screens.*

9. What kind of code generally belongs in `home.js` (a leaf screen file) rather than a `_layout.js`?

- A) The screen's own specific UI content B) App-wide Context Providers C) The top-level `Stack` navigator D) Global font loading

Answer: A — *Leaf screens should just render their own specific UI.*

10. Why might a tab bar's appearance be defined in `app/(tabs)/_layout.js` rather than the root `_layout.js`?

- A) Because the tab bar specifically only applies to that nested section of the app B) Because the root layout can't return JSX C) Because tabs aren't supported in Expo Router D) Because of file size limits

Answer: A — *Section-specific layout concerns belong in that section's own layout file.*

11. What's the benefit of nesting layouts (root → section → screen)?

- A) Each level can add its own shared structure without repeating code at every screen B) It makes the app slower C) It's required by JavaScript syntax D) It disables routing

Answer: A — *This avoids duplicating shared setup code across many individual screens.*

12. If you need fonts loaded before anything renders, where would you typically put that loading logic?

- A) Inside every individual screen B) In the root `_layout.js`, since it runs first for the whole app C) In a CSS file D) It can't be done in Expo Router

Answer: B — *Loading critical resources once at the root avoids repeating it everywhere.*

13. Which is true about Context Providers placed in the root layout?

- A) They cascade down to all nested screens automatically B) They only apply to the root layout file itself C) They must be re-declared in every screen D) They disable navigation

Answer: A — *React Context naturally flows down through the entire component tree.*

14. Why is it considered bad practice to define a Stack navigator inside a regular screen component?

- A) Because navigators belong in `_layout.js` files that define structure for a route segment, not inside individual screens B) Because Stack doesn't exist in Expo Router C) Because screens can't import anything D) There's no issue with this

Answer: A — *Mixing navigator setup into a leaf screen breaks the intended separation of concerns.*

True / False (7)

1. The root `_layout.js` is the first layout the entire app passes through.

Answer: True — *All navigation paths start from the root layout.*

2. Nested folders cannot have their own separate `_layout.js` files.

Answer: False — *Nested folders can define their own scoped layout files.*

3. Providers like `AuthProvider` placed in the root layout are automatically available to all nested screens.

Answer: True — *Context cascades down through the whole component tree.*

4. A regular screen file like `home.js` should typically define the app's main Stack navigator.

Answer: False — *Top-level navigators belong in a `_layout.js` file, not a leaf screen.*

5. A tab bar's configuration is often defined in a nested `(tabs)/_layout.js` rather than the root layout.

Answer: True — *Section-specific layout concerns are scoped to that section's own layout.*

6. Defining global providers separately inside every screen is the recommended approach instead of using the root layout.

Answer: False — *A single root-level provider is the recommended, more maintainable approach.*

7. Nested layouts allow each section of the app to add its own shared structure without repeating it on every screen.

Answer: True — *That's the main benefit of the layout hierarchy.*

Code Correction (4)

1. `app/(tabs)/home.js` containing `<AuthProvider><Stack>...</Stack></AuthProvider>`

Fix: Move that logic into `app/_layout.js`, and let `home.js` just return its own screen UI
Providers and top-level navigators belong in a layout file, not inside an individual screen.

2. `SafeAreaProvider` duplicated inside every screen file separately

Fix: A single `<SafeAreaProvider>` wrapping everything in the root `_layout.js`
One provider at the root cascades to all nested screens, so repeating it per-screen is unnecessary.

3. Tab bar styling code placed directly inside the root `_layout.js`, affecting unrelated screens

Fix: Move tab-specific layout/styling into `app/(tabs)/_layout.js`
Section-specific layout concerns belong in that section's own nested layout file.

4. A screen re-importing and re-wrapping `AuthProvider` "just to make sure" auth works

Fix: Simply call `useAuth()` inside the screen – no need to re-wrap anything
Context cascades automatically to all nested components; re-wrapping providers per screen is redundant.

16. Styling: StyleSheet.create()

Multiple Choice Questions (14)

1. Why is StyleSheet.create() preferred over inline style objects?

- A) It's required by Expo B) It improves performance and can validate style properties C) It removes the need for the style prop D) It makes text bold automatically

Answer: B — *It registers styles efficiently and can catch invalid property names.*

2. How do you apply two styles to one component?

- A) style={styles.a + styles.b} B) style={[styles.a, styles.b]} C) style="a b" D) Not possible

Answer: B — *The style prop accepts an array, merging styles in order.*

3. Are inline style objects like style={{padding:10}} recreated on every render?

- A) Yes B) No, they're cached automatically C) Only on Android D) Only with class components

Answer: A — *Each render creates a brand-new object literal in memory.*

4. What does StyleSheet.create() return for each style key?

- A) A CSS class name string B) A reference/ID React Native can use efficiently C) A Promise D) A function

Answer: B — *It returns optimized references rather than plain raw objects.*

5. When combining styles in an array, which one wins if both define the same property?

- A) The first one always B) The later one in the array overrides the earlier one for that property C) Neither applies D) It throws an error

Answer: B — *Styles are merged in order, with later entries taking precedence.*

6. Which of these is a valid StyleSheet.create() usage?

- A) StyleSheet.create({container: {flex:1}}) B) StyleSheet.create([flex:1]) C) StyleSheet({flex:1}) D) new StyleSheet({flex:1})

Answer: A — *It takes an object whose keys are named style definitions.*

7. Why might a typo like "colour" instead of "color" be caught more easily with StyleSheet.create()?

- A) Because it validates against RN's known style properties B) Because it auto-corrects spelling C) Because it's written in British English mode D) It isn't caught at all, ever

Answer: A — *It can flag property names that don't match valid RN style keys.*

8. What's the main "Style Sheet" concept borrowed from web CSS, in RN?

- A) Linking external .css files directly B) Defining grouped style rules as JS objects, applied via the style prop C) Using <style> tags inside JSX D) Importing Bootstrap

Answer: B — *RN mirrors the idea of grouped, reusable style rules using JS objects.*

9. Which approach better separates UI structure from styling logic?

- A) Inline objects scattered through JSX B) Centralized StyleSheet.create() definitions C) No styling at all D) Styling via comments

Answer: B — *Keeping styles in one place makes the JSX easier to read.*

10. What happens performance-wise if you use inline objects extensively across many re-renders?

- A) No measurable difference ever B) New objects get created repeatedly, adding minor overhead compared to reused StyleSheet references C) The app becomes faster D) It disables re-rendering

Answer: B — *Repeated object creation adds unnecessary overhead at scale.*

11. Can you reuse a single styles.container object across multiple different components?

- A) No, styles are component-specific only B) Yes, the same style object can be applied wherever needed C) Only with Redux D) Only in class components

Answer: B — *Style definitions are just plain objects that can be imported and reused anywhere.*

12. Which best demonstrates combining a base style with an optional conditional style?

- A) style={[styles.box, isActive && styles.activeBox]} B) style={styles.box + isActive} C) style="box active" D)

style={isActive ? "active" : "box"}

Answer: A — *Arrays let you conditionally include extra styles cleanly.*

13. Is StyleSheet.create() required for an app to function at all?

- A) Yes, the app won't run without it B) No, it's a best practice for performance/organization, not a hard requirement C) Only required on iOS D) Only required for images

Answer: B — *Inline styles still work; StyleSheet.create() is simply the recommended convention.*

14. What's a downside of writing all styles directly inline inside JSX for a large app?

- A) Harder to read, no validation, and objects recreated on every render B) It runs faster C) It removes all bugs D) There is no downside

Answer: A — *At scale, inline styles hurt both readability and performance.*

True / False (7)

1. StyleSheet.create() helps catch invalid style property names early.

Answer: True — *It validates property names against RN's known style schema.*

2. Inline style objects are reused between renders rather than recreated.

Answer: False — *A new object literal is created on every single render.*

3. The style prop can accept an array of multiple style objects.

Answer: True — *Arrays let you merge several style objects together.*

4. Later styles in a combined style array override earlier ones for matching properties.

Answer: True — *Styles are applied in array order, with later entries taking precedence.*

5. StyleSheet.create() is mandatory for a React Native app to render anything at all.

Answer: False — *Plain inline style objects also work; it's a best practice, not a requirement.*

6. Centralizing styles with StyleSheet.create() makes JSX markup easier to read.

Answer: True — *Separating styles from markup keeps the JSX focused on structure.*

7. There is no performance difference at all between inline objects and StyleSheet.create() at scale.

Answer: False — *Repeated object creation from inline styles does add minor overhead at scale.*

Code Correction (4)

1. <View style={{padding:16, backgroundColor:'#fff'}}> repeated inline across many components

Fix: const styles = StyleSheet.create({container:{padding:16, backgroundColor:'#fff'}}); ...
style={styles.container}

Centralizing the style avoids recreating the same object on every render and improves readability.

2. style={styles.box + styles.shadow}

Fix: style={[styles.box, styles.shadow]}

Multiple styles must be combined using an array, not the + operator.

3. const styles = StyleSheet.create([flex:1]);

Fix: const styles = StyleSheet.create({container:{flex:1}});

StyleSheet.create() expects an object of named style keys, not an array.

4. <Text style="fontSize: 18; fontWeight: bold;">Hello</Text>

Fix: <Text style={{fontSize:18, fontWeight:'bold'}}>Hello</Text>

React Native styles are JavaScript objects, not CSS-style strings like on the web.

17. Async/Await vs. Promise Chains

Multiple Choice Questions (14)

1. Which keyword must precede a function for await to be usable inside it?

- A) defer B) async C) yield D) wait

Answer: B — *await is only valid inside functions explicitly declared async.*

2. What replaces .catch() for error handling in async/await code?

- A) .error() B) try/catch C) .finally() only D) Nothing, errors disappear

Answer: B — *try/catch is the standard error-handling structure for async/await.*

3. What does await do to a Promise inside an async function?

- A) Cancels it B) Pauses that function until the Promise resolves (or rejects) C) Runs it twice D) Converts it to a callback

Answer: B — *It suspends execution of that function until the awaited Promise settles.*

4. Why is async/await often considered more readable than chained .then() calls?

- A) It reads top-to-bottom like synchronous code B) It removes the need for any error handling C) It runs faster on the CPU D) It avoids using Promises entirely

Answer: A — *Sequential await lines look like ordinary step-by-step code.*

5. Can you use await outside of an async function?

- A) Yes, always B) No, it causes a SyntaxError C) Only inside loops D) Only inside classes

Answer: B — *The parser only permits await inside functions marked async.*

6. What happens to code after an await line within an async function, until the awaited Promise settles?

- A) It runs immediately regardless B) It waits, effectively pausing that function's execution C) It throws an error always D) It skips ahead

Answer: B — *Execution of that function pauses until the awaited value is ready.*

7. Which of these correctly wraps a risky awaited call to handle errors?

- A) try { await x(); } catch(e) { } B) catch { await x(); } try C) await try { x(); } D) x().try().catch()

Answer: A — *The await call goes inside try, with errors caught in the matching catch block.*

8. What does an async function always return, even if you write a plain return value inside it?

- A) The raw value directly, never wrapped B) A Promise that resolves to that value C) undefined always D) A callback

Answer: B — *Every async function's result is automatically wrapped in a Promise.*

9. Why does async/await help avoid "callback hell"?

- A) Multiple dependent async steps can be written as sequential await lines instead of deeply nested callbacks B) It deletes all callbacks from JS C) It disables Promises D) It requires fewer functions overall, always

Answer: A — *Sequential awaits replace the need for deeply nested callback structures.*

10. What's a typical structure for handling multiple sequential API calls with async/await?

- A) Multiple nested .then() chains B) Several consecutive await lines inside one try block C) A single callback with five arguments D) A while loop only

Answer: B — *Each step is simply written one after another, all inside one try block.*

11. If an awaited call rejects inside a try block, what happens?

- A) The function crashes the whole app B) Control jumps to the matching catch block C) It's silently ignored always D) It retries automatically forever

Answer: B — *A rejection is treated like a thrown error, caught by the surrounding catch.*

12. Which is functionally closer to: axios.get(url).then(res=>console.log(res.data))?

- A) const res = await axios.get(url); console.log(res.data); (inside an async function) B) axios.get(url); console.log(res.data); C) async axios.get(url).data; D) await console.log(axios.get(url));

Answer: A — *Both achieve the same result — logging the data once the request resolves.*

13. Why might debugging be easier with async/await compared to long .then() chains?

A) Stepping through await lines feels like stepping through normal sequential code B) Debuggers don't support Promises at all C) Async/await disables breakpoints D) There's no real difference

Answer: A — *Sequential code is generally more intuitive to step through than nested callbacks.*

14. What's the correct way to declare an async arrow function?

A) `const fn = async () => { }` B) `const fn = () => async { }` C) `async const fn = () => { }` D) `const async fn = () => { }`

Answer: A — *The async keyword comes immediately before the parameter list.*

True / False (7)

1. The await keyword can only be used inside a function declared with async.

Answer: True — *This is a strict syntax requirement enforced by the parser.*

2. async/await completely eliminates the need for any error handling.

Answer: False — *You still need try/catch to handle rejected Promises.*

3. An async function always returns a Promise, even if you return a plain value inside it.

Answer: True — *The returned value is automatically wrapped in a resolved Promise.*

4. Multiple sequential await calls inside one try block can replace deeply nested .then() chains.

Answer: True — *This is exactly how async/await avoids callback hell.*

5. Using await pauses the entire application, not just the function it's in.

Answer: False — *Only the specific async function pauses; the rest of the app keeps running.*

6. try/catch is the standard way to handle errors in async/await code.

Answer: True — *It mirrors how synchronous error handling normally works.*

7. await can be used safely inside a regular function as long as it eventually calls an async function.

Answer: False — *The function containing the await itself must be declared async.*

Code Correction (4)

1. `function getData() { const res = await axios.get(url); console.log(res.data); }`

Fix: `async function getData() { const res = await axios.get(url); console.log(res.data); }`
await is only valid inside a function explicitly marked async.

2. `async function getData() { const res = await axios.get(url); console.log(res.data); } (no error handling)`

Fix: Wrap the body in `try { ... } catch (err) { console.log(err); }`
Without try/catch, a rejected Promise inside an async function would throw unhandled.

3. `const fn = () => async { return 5; };`

Fix: `const fn = async () => { return 5; };`
The async keyword must come before the parameter list, not inside the function body.

4. Assuming async function `getData(){ return 5; }` returns the number 5 directly when called

Fix: `getData().then(value => console.log(value));` // since it returns a Promise resolving to 5
Any async function always returns a Promise, so its result must be unwrapped with await or .then().